

```
In [2]: # Experiment No. 10
# Fashion-MNIST Image Classification using CNN

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 1. Load Dataset
df = pd.read_csv("fashion-mnist_test.csv")

X = df.drop("label", axis=1).values
y = df["label"].values

# 2. Preprocessing
X = X.astype("float32") / 255.0
X = X.reshape(-1, 28, 28, 1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("Dataset Loaded Successfully")
print("Training:", X_train.shape)
print("Testing :", X_test.shape)

# 3. CNN Model
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation="relu",
                           input_shape=(28,28,1)),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Conv2D(64, (3,3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

# 4. Compile and Train
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)
```

```

# 5. Prediction
pred = np.argmax(model.predict(X_test, verbose=0), axis=1)

# 6. Evaluation
print("\nPerformance Metrics")
print("Accuracy :", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted"))
print("Recall   :", recall_score(y_test, pred, average="weighted"))
print("F1-Score :", f1_score(y_test, pred, average="weighted"))

# 7. Confusion Matrix
plt.figure(figsize=(7,6))
plt.imshow(confusion_matrix(y_test, pred))
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()

# 8. Accuracy Graph
plt.plot(history.history["accuracy"], label="Training")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# 9. Sample Predictions
plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_test[i].reshape(28,28), cmap="gray")
    plt.title(f"Actual: {y_test[i]}\nPredicted: {pred[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()

print("\nExperiment No. 10 Completed Successfully")

```

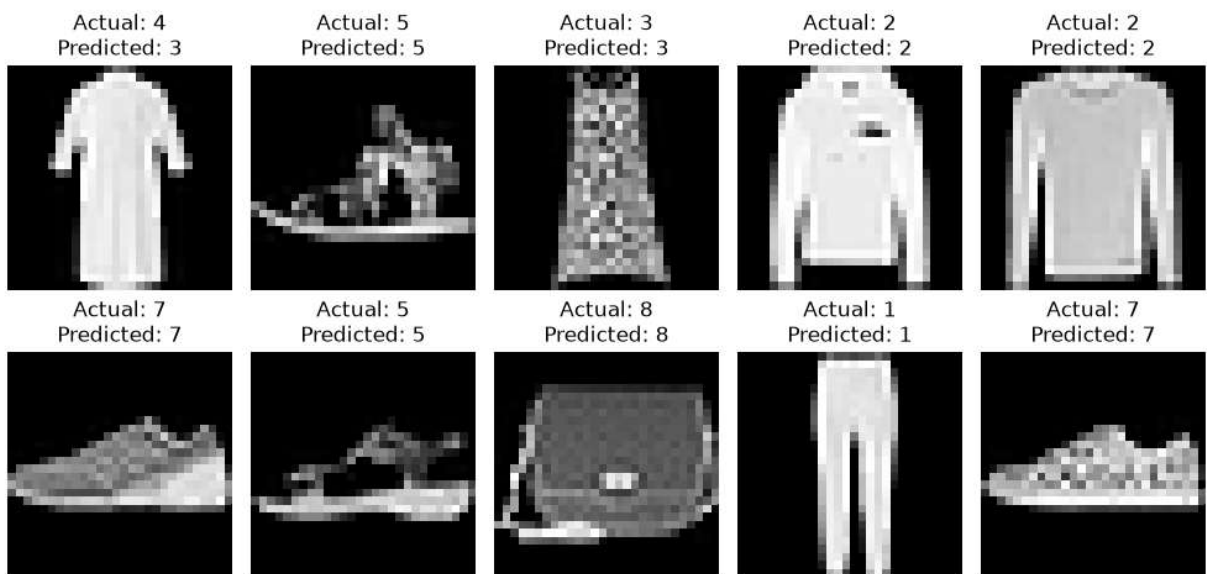
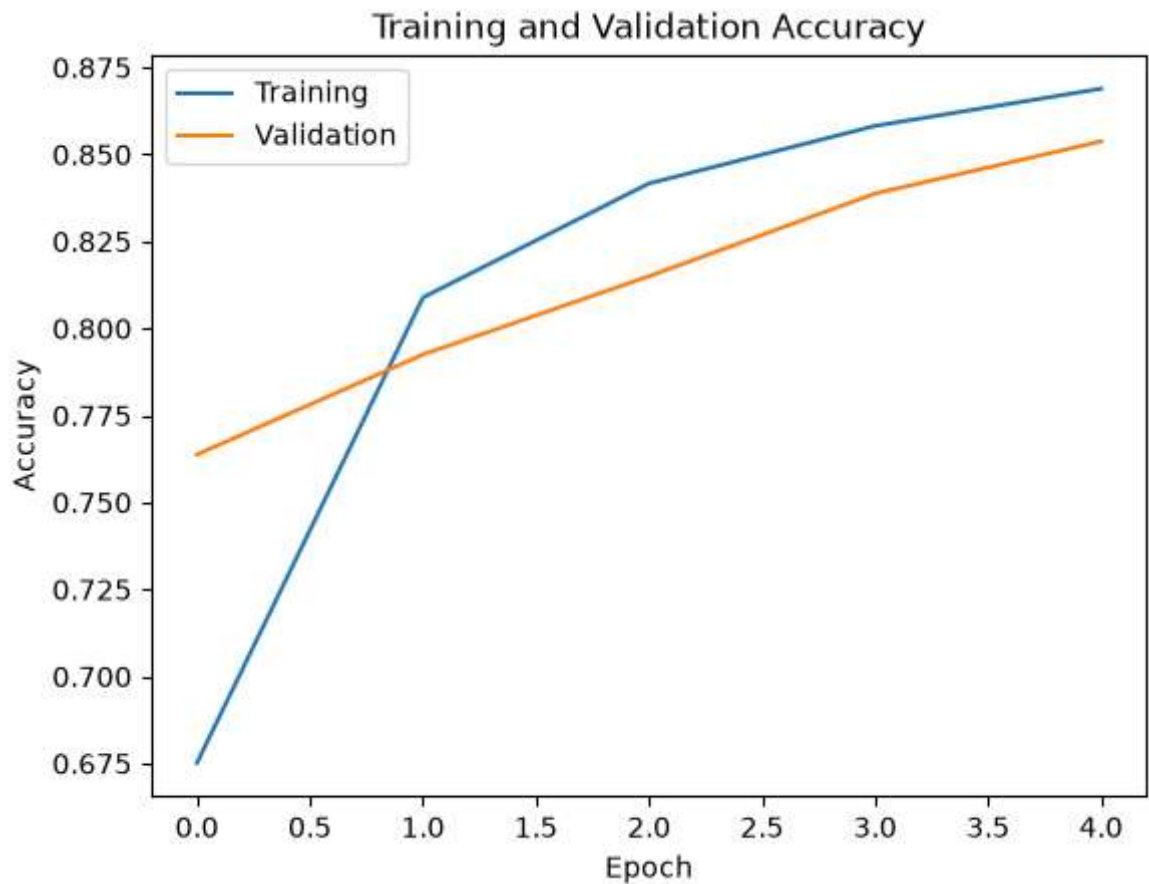
Dataset Loaded Successfully

Training: (8000, 28, 28, 1)

Testing : (2000, 28, 28, 1)

C:\Users\Himanshu\AppData\Roaming\Python\Python312\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Experiment No. 10 Completed Successfully

Experiment No. 10 : Digit Classification using CNN Experiment Name : Digit Classification
 using CNN Aim : To classify handwritten digits using CNN. Model : CNN Dataset : MNIST
 Objective : To train and evaluate a digit classification model.

QUESTIONS AND ANSWERS

Q1. What is image classification? How does it differ from object detection and image segmentation?

Answer: Image classification assigns a label to an image. Object detection identifies and locates objects using bounding boxes. Image segmentation classifies individual pixels of an image.

Q2. Explain the architecture and working principle of a CNN.

Answer: A CNN consists of convolution, pooling, and fully connected layers. It automatically learns image features and uses them to classify images.

Q3. What is the role of Convolutional, Pooling, and Fully Connected Layers?

Answer: Convolutional layers extract features, pooling layers reduce image size, and fully connected layers perform final classification.

Q4. Why is image normalization performed?

Answer: Normalization scales pixel values to a suitable range, usually 0 to 1. It helps the model train faster and more effectively.

Q5. Explain ReLU and Softmax.

Answer: ReLU adds non-linearity and helps the network learn complex features. Softmax converts the final outputs into class probabilities.

Q6. What is a Confusion Matrix?

Answer: A Confusion Matrix is a table that shows correct and incorrect predictions for different classes. It helps evaluate classification performance.

Q7. Differentiate Accuracy, Precision, Recall, and F1-Score.

Answer: Accuracy measures overall correct predictions. Precision measures correct positive predictions. Recall measures correctly detected actual positives. F1-Score is the balance between Precision and Recall.

Q8. Compare SIFT/HOG + SVM with CNN.

Answer: SIFT/HOG + SVM requires manual feature extraction, while CNN automatically learns useful features from images. CNNs generally work better with large datasets.

Q9. Mention five applications of CNN-based image classification.

Answer:

1. Handwritten digit recognition
2. Face recognition
3. Medical image analysis
4. Object/image categorization
5. Document classification

Q10. How can Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve performance?

Answer: Data Augmentation increases training data variety. Transfer Learning uses knowledge from a pre-trained model. Hyperparameter Tuning helps find better training settings and improve model performance.